

Gradient Descent and Moving Beyond Linearity

Week 4

Learning Outcomes

By the end of this week, you should be able to:

- Explain how gradient descent minimizes an objective function and describe the role of the learning rate.
- Explain why convexity is useful for optimization.
- Describe how polynomial regression, step functions, and splines can model nonlinear relationships.
- Compare kNN, weighted kNN, and local regression as local modelling approaches.
- Explain how generalized additive models extend nonlinear modelling to multiple predictors.

Review: Regularized Linear Regression

- Ordinary least squares can perform poorly when p is large relative to n .
 - i.e. estimates can have high variance
- Ridge and the lasso can improve test performance by:
 - reducing variance;
 - accepting some additional bias.
- The benefit can be substantial when the true coefficients are **small** or **sparse**.

If only $s \ll p$ features are predictive, we would ideally fit the model using only those s features.

Objective Functions

So far, we have defined our regression estimates by asking them to **minimize an objective function**.

For ordinary least squares:

$$\min_{\beta} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2$$

For ridge regression:

$$\min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\}$$

For the Lasso:

$$\min_{\boldsymbol{\beta}} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p |\beta_j| \right\}$$

A General Minimization Problem

Suppose we want to solve

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w} \in \Theta} \mathcal{J}(\mathbf{w}; \mathcal{D}^{\text{train}}) := \arg \min_{\mathbf{w} \in \Theta} \mathcal{J}(\mathbf{w}).$$

where

- $\mathcal{J}(\mathbf{w})$ is a differentiable function in $\mathbf{w} = (w_1, \dots, w_p)$;
- Θ is the parameter space for $\mathbf{w}$, typically a subset of $\mathbb{R}^p$.

The optimal solution, if it exists in the interior, must occur at a **critical point** i.e. point to which the derivative is zero (partial derivatives to zero for multi-dimensional parameter)

Finding the Optimal Solution

A **partial derivative** measures how a multivariable function changes with respect to one argument while holding the others fixed.

For example,

$$\frac{\partial}{\partial w_1} \mathcal{J}(w_1, w_2) = \lim_{h \rightarrow 0} \frac{\mathcal{J}(w_1 + h, w_2) - \mathcal{J}(w_1, w_2)}{h}.$$

The minimum must occur at a point where the partial derivatives are zero

$$\nabla \mathcal{J}(\mathbf{w}) = \begin{bmatrix} \frac{\partial \mathcal{J}}{\partial w_1} \\ \vdots \\ \frac{\partial \mathcal{J}}{\partial w_p} \end{bmatrix} = \mathbf{0}.$$

This implies a system of equations, which we can solve analytically in some scenarios.

We may also use optimization techniques that iteratively get us closer to the solution.

Direct Solution: OLS

For ordinary least squares,

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w} \in \mathbb{R}^p} \mathcal{J}(\mathbf{w}) = \arg \min_{\mathbf{w} \in \mathbb{R}^p} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2.$$

The partial derivatives w.r.t $\mathbf{w}$ are

$$\frac{\partial \mathcal{J}}{\partial \mathbf{w}} = -2\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}).$$

(If not familiar with multi-dimensional derivatives, calculate $\frac{\partial \mathcal{J}}{\partial w_j}$ and stack them together)

Setting this equal to zero gives

$$\mathbf{X}^\top \mathbf{X} \hat{\mathbf{w}} = \mathbf{X}^\top \mathbf{y},$$

so, when the inverse exists:

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Direct Solution: Ridge

For ridge regression,

$$\hat{\mathbf{w}}_{\lambda}^R = \arg \min_{\mathbf{w} \in \mathbb{R}^p} \left\{ \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_2^2 \right\}.$$

The partial derivatives w.r.t $\mathbf{w}$ are

$$\frac{\partial \mathcal{J}}{\partial \mathbf{w}} = -2\mathbf{X}^{\top}(\mathbf{y} - \mathbf{X}\mathbf{w}) + 2\lambda\mathbf{w}.$$

Setting this equal to zero gives

$$(\mathbf{X}^{\top}\mathbf{X} + \lambda\mathbf{I}_p)\hat{\mathbf{w}}_{\lambda}^R = \mathbf{X}^{\top}\mathbf{y},$$

and therefore

$$\boxed{\hat{\mathbf{w}}_{\lambda}^R = (\mathbf{X}^{\top}\mathbf{X} + \lambda\mathbf{I}_p)^{-1}\mathbf{X}^{\top}\mathbf{y}}$$

Gradient Descent

Now consider a second way to solve

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \mathcal{J}(\mathbf{w}).$$

which is more broadly applicable: **gradient descent**.

Many times, we do **not** have a convenient direct solution to

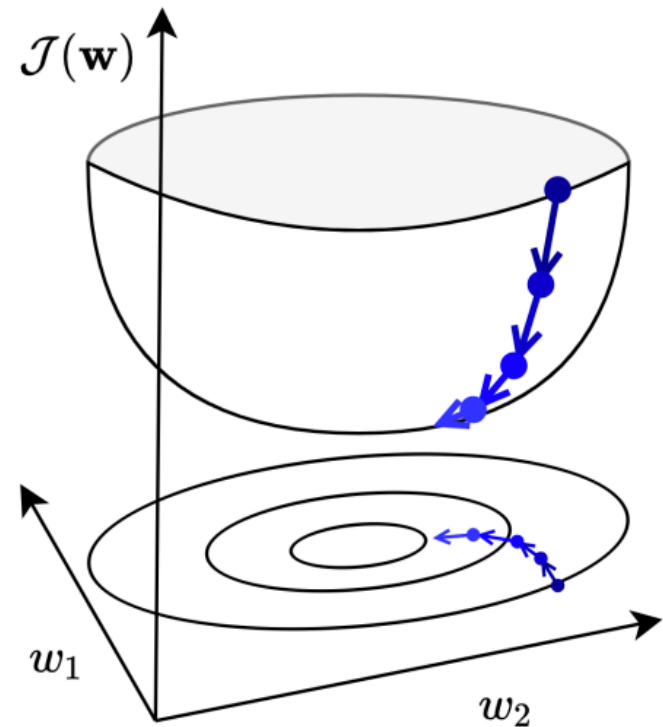
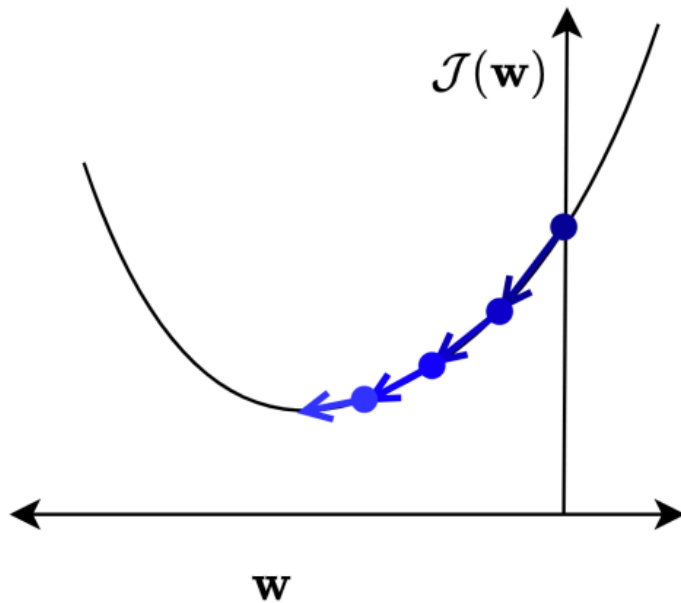
$$\frac{\partial \mathcal{J}}{\partial \mathbf{w}} = 0.$$

Gradient descent is an **iterative algorithm**.

Rather than solving directly for the optimum, we repeatedly update the parameters so that we move toward a smaller value of the objective.

Gradient Descent

We initialize $\mathbf{w}$ at some reasonable starting value, (e.g. all zeros) and repeatedly adjust it in the direction of **steepest descent** of the loss function $\mathcal{J}$.



What is the direction of the steepest descent of $\mathcal{J}(\mathbf{w})$ at $\mathbf{w}$?

Gradient Descent

By definition, the direction of the greatest increase in $\mathcal{J}(\mathbf{w})$ at $\mathbf{w}^{(0)}$ is its **gradient**:

$$\left. \frac{\partial \mathcal{J}(\mathbf{w})}{\partial \mathbf{w}} \right|_{\mathbf{w}=\mathbf{w}^{(0)}} \in \mathbb{R}^p$$

So, we update $\mathbf{w}$ in the **opposite** direction of the gradient at $\mathbf{w}^{(0)}$:

$$\mathbf{w}^{(1)} = \mathbf{w}^{(0)} - \alpha \left. \frac{\partial \mathcal{J}(\mathbf{w})}{\partial \mathbf{w}} \right|_{\mathbf{w}=\mathbf{w}^{(0)}}$$

for some $\alpha > 0$.

If α is chosen small, then

$$\mathcal{J}(\mathbf{w}^{(1)}) < \mathcal{J}(\mathbf{w}^{(0)})$$

unless $\left. \frac{\partial \mathcal{J}(\mathbf{w})}{\partial \mathbf{w}} \right|_{\mathbf{w}^{(0)}}$ is zero.

Gradient Descent: Coordinatewise View

For iterations $k = 0, 1, 2, \dots$, and for each $j \in \{1, \dots, p\}$,

$$w_j^{(k+1)} \leftarrow w_j^{(k)} - \alpha \frac{\partial \mathcal{J}}{\partial w_j} \bigg|_{\mathbf{w} = \mathbf{w}^{(k)}}$$

The **learning rate** (or step size), α , controls the size of each update.

- larger α : parameters change more quickly;
- smaller α : parameters change more slowly.

Typical values may be small, such as 0.01 or 0.0001, although the appropriate scale depends on the problem.

Gradient Descent for OLS

For OLS,

$$\mathcal{J}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2.$$

Update rule in vector form at the $(k + 1)$ th iteration:

$$\begin{aligned}\mathbf{w}^{(k+1)} &\leftarrow \mathbf{w}^{(k)} - \alpha \frac{\partial \mathcal{J}}{\partial \mathbf{w}} \bigg|_{\mathbf{w}=\mathbf{w}^{(k)}} \\ &= \mathbf{w}^{(k)} + 2\alpha \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}^{(k)}).\end{aligned}$$

Initialization: $\mathbf{w}^{(0)} = \mathbf{0}$

Gradient Descent for Ridge

For ridge regression,

$$\mathcal{J}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2.$$

Update rule in vector form at the $(k + 1)$ th iteration:

$$\begin{aligned}\mathbf{w}^{(k+1)} &\leftarrow \mathbf{w}^{(k)} - \alpha \frac{\partial \mathcal{J}}{\partial \mathbf{w}} \bigg|_{\mathbf{w}=\mathbf{w}^{(k)}} \\ &= \mathbf{w}^{(k)} + 2\alpha \mathbf{X}^\top \left(\mathbf{y} - \mathbf{X}\mathbf{w}^{(k)} \right) - 2\alpha \lambda \mathbf{w}^{(k)}.\end{aligned}$$

Initialization: $\mathbf{w}^{(0)} = \mathbf{0}$

The final term is the contribution from the ridge penalty and pulls the coefficients toward zero.

What About the Lasso?

For the lasso,

$$\mathcal{J}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_1.$$

The difficulty is that

$$\|\mathbf{w}\|_1 = \sum_{j=1}^p |w_j|$$

is **not differentiable at** $w_j = 0$.

So ordinary gradient descent cannot be applied in exactly the same way at those points.

Subgradients

For convex functions that are not differentiable everywhere, we can use a **subgradient** at $\mathbf{x}$, defined as:

$$\partial \mathcal{J}(\mathbf{x}) := \{v : \mathcal{J}(\mathbf{y}) \geq \mathcal{J}(\mathbf{x}) + (\mathbf{y} - \mathbf{x})^\top v, \quad \forall \mathbf{y} \in \Theta\}.$$

For $f(x) = |x|$,

$$\partial f(x) = \begin{cases} 1, & x > 0, \\ -1, & x < 0, \\ (-1, 1), & x = 0. \end{cases}$$

For $f(\mathbf{x}) = \|\mathbf{x}\|_1$,

$$[\partial f(\mathbf{x})]_j = \begin{cases} 1, & x_j > 0, \\ -1, & x_j < 0, \\ (-1, 1), & x_j = 0. \end{cases}$$

This gives us a generalized notion of “slope” even at the kink at zero.

Subgradient Descent for the Lasso

For the Lasso,

$$\mathcal{J}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_1.$$

Update rule in vector form at the $(k + 1)$ th iteration:

$$\begin{aligned}\mathbf{w}^{(k+1)} &\leftarrow \mathbf{w}^{(k)} - \alpha \frac{\partial \mathcal{J}}{\partial \mathbf{w}} \bigg|_{\mathbf{w}=\mathbf{w}^{(k)}} \\ &= \mathbf{w}^{(k)} + 2\alpha \mathbf{X}^\top \left(\mathbf{y} - \mathbf{X}\mathbf{w}^{(k)} \right) - \alpha \lambda \partial (\|\mathbf{w}\|_1) \big|_{\mathbf{w}^{(k)}}.\end{aligned}$$

Initialization: $\mathbf{w}^{(0)} = \mathbf{0}$

Stopping Criteria

Because gradient descent is iterative, we need a **stopping criterion**.

We might stop when:

1. the objective value barely changes,

$$\left| \mathcal{J}(\mathbf{w}^{(k+1)}) - \mathcal{J}(\mathbf{w}^{(k)}) \right| \leq \varepsilon;$$

2. the parameters barely change,

$$\|\mathbf{w}^{(k+1)} - \mathbf{w}^{(k)}\|_2 \leq \varepsilon;$$

...

3. or we reach a maximum number of iterations, such as $M = 1000$.

When Do We Expect Gradient Descent to Work?

Recall that we are trying to solve

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w} \in \Theta} \mathcal{J}(\mathbf{w}).$$

- Obviously, $\mathcal{J}$ should be differentiable.
- If $\mathcal{J}$ is also **convex**, and Θ is a convex set, then gradient descent with a suitable choice of step size guarantees to find the optimal solution.
- In many cases, $\Theta = \mathbb{R}^p$, which is convex.

Convex Sets

A set S is **convex** if for any $\mathbf{x}_0, \mathbf{x}_1 \in S$,

$$(1 - \lambda)\mathbf{x}_0 + \lambda\mathbf{x}_1 \in S \quad \text{for every } 0 \leq \lambda \leq 1.$$

In words:

If you draw a straight line between any two points in the set, the entire line segment stays inside the set.

The Euclidean space $\mathbb{R}^p$ is convex.

Convex Functions

A function f is **convex** if for any x_0, x_1 in its domain,

$$f((1 - \lambda)x_0 + \lambda x_1) \leq (1 - \lambda)f(x_0) + \lambda f(x_1)$$

for every $\lambda \in [0, 1]$.

Intuitively, a convex function is **bowl-shaped**.

Why do we care?

For a convex objective, every local minimum is also a global minimum.

How Can We Tell if a Loss is Convex?

1. We can verify the definition directly.

2. If f is twice differentiable and

$$f''(x) \geq 0$$

for all x , then f is convex.

For example, the least squares loss function $(y - t)^2$ is convex as a function of t

Likewise, the loss function in logistic regression

$$-yt + \log(1 + e^t)$$

is convex in t .

How Can We Tell if a Loss is Convex?

3. **Linear transformations preserve convexity** in many of the forms we use in statistical learning.

If f is a convex function and g is a linear function, then both $f \circ g$ and $g \circ f$ are convex.

For example, the least squares loss $(y - \mathbf{x}^\top \mathbf{w})^2$ is convex as a function of $\mathbf{w}$.

Likewise, the negative log-likelihood under logistic regression,

$$-y\mathbf{x}^\top \mathbf{w} + \log \left(1 + e^{\mathbf{x}^\top \mathbf{w}} \right),$$

is convex in $\mathbf{w}$.

Therefore sums of these losses over observations are also convex.

Why Gradient Descent for Linear Regression?

The linear regression objective

$$\sum_{i=1}^n (y_i - \mathbf{x}_i^\top \mathbf{w})^2$$

is convex.

So why use gradient descent when a closed-form solution exists?

When p is large, directly computing

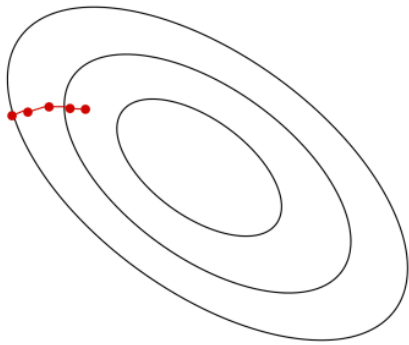
$$(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

may be expensive.

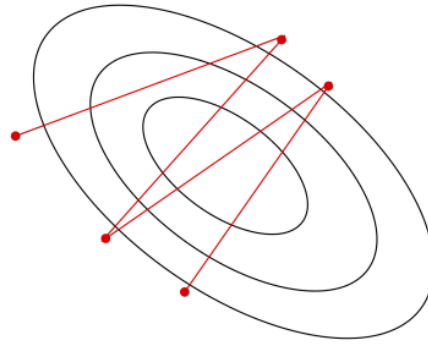
A gradient descent update can be computationally attractive because it avoids explicitly inverting a large matrix.

The Learning Rate Matters

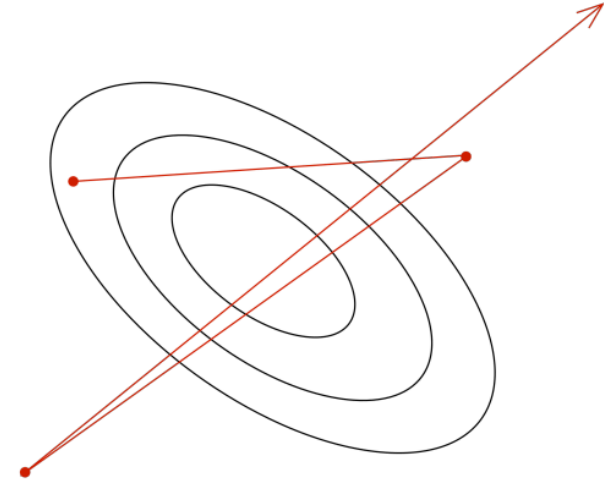
The learning rate α is a **hyperparameter** that we need to tune. Its value can dramatically affect optimization.



α too small:
slow progress



α too large:
oscillations



α much too large:
instability

We often try several candidate learning rates and examine their optimization behaviour.

Training Curves

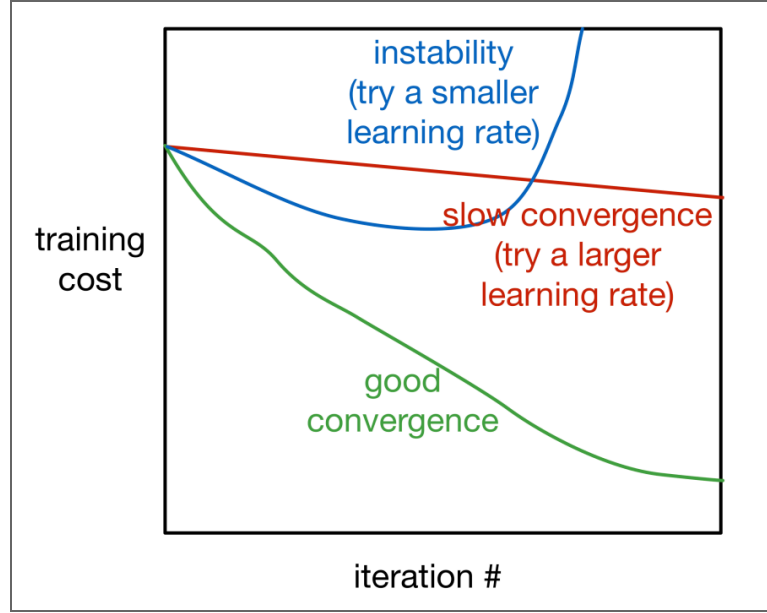
A useful diagnostic is to look at the **training cost**: plot the training cost as a function of iteration.

Training curves can help identify:

- very slow convergence;
- unstable updates;
- a learning rate that may be too small or too large.

Important: the training cost could be used to check whether the optimization problem reaches certain convergence. But:

- It does not tell whether we reach the global minimum or not
- It does not tell us anything on the performance of the fitted model



Batch Gradient Descent

Recall for OLS that:

$$\hat{\mathbf{w}}^{(k+1)} = \hat{\mathbf{w}}^{(k)} + \alpha \sum_{i=1}^n \left[y_i - \mathbf{x}_i^\top \hat{\mathbf{w}}^{(k)} \right] \mathbf{x}_i.$$

Computing the gradient requires summing over **all** of the training examples, which can be done via matrix / vector operations.

The fact that it uses all training samples is known as **batch training**.

Stochastic Gradient Descent

Batch gradient descent can be expensive when n is extremely large.

Stochastic gradient descent (SGD) uses only one randomly selected training example for each update.

At iteration k :

1. choose $i \in \{1, \dots, n\}$ at random;
2. update using only observation i .

For OLS,

$$\hat{\mathbf{w}}^{(k+1)} = \hat{\mathbf{w}}^{(k)} + \alpha \left[y_i - \mathbf{x}_i^\top \hat{\mathbf{w}}^{(k)} \right] \mathbf{x}_i$$

Why Use SGD?

Pros

- Computational cost of each SGD update is independent of n !
- SGD can make significant progress before even seeing all the data!
- Mathematical justification: the gradients between SGD and GD have the same expectation for i.i.d. data.

Cons

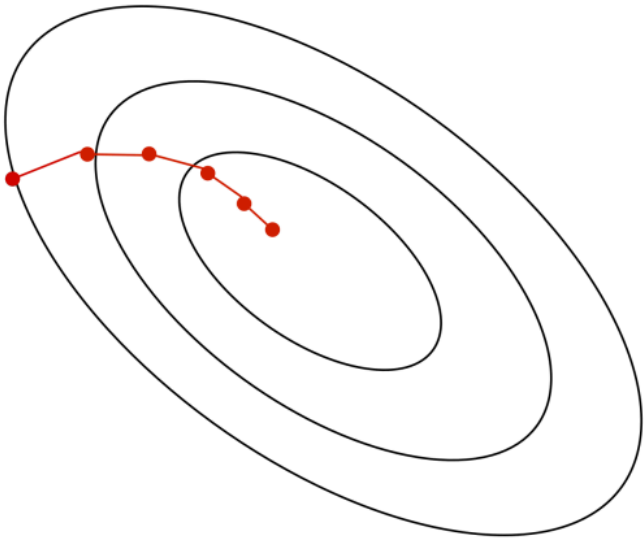
- We are using only a single training example to estimate the gradient, so the variance in the estimate may be high.

Compromise Approach

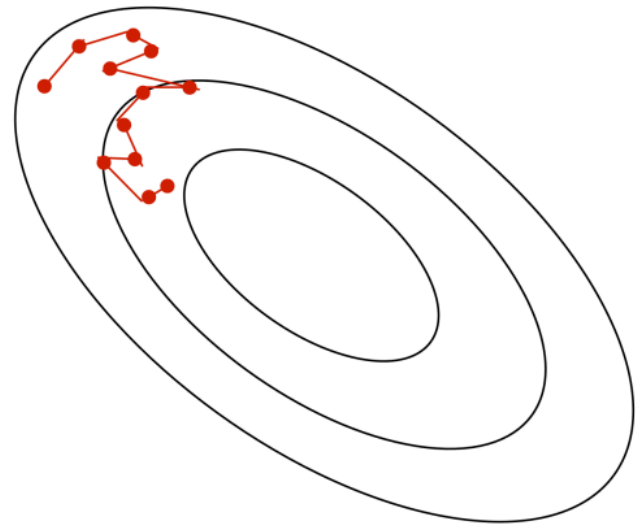
- compute the gradients on a randomly chosen medium-sized set of training examples $\mathcal{M} \subset \{1, \dots, n\}$, called a **mini-batch**
- Stochastic gradients computed on larger mini-batches have smaller variance.
- The mini-batch size $|\mathcal{M}|$ is a hyperparameter that needs to be set

Stochastic Gradient Descent

- Batch gradient descent moves directly downhill
- SGD takes steps in a noisy direction, but moves downhill on average



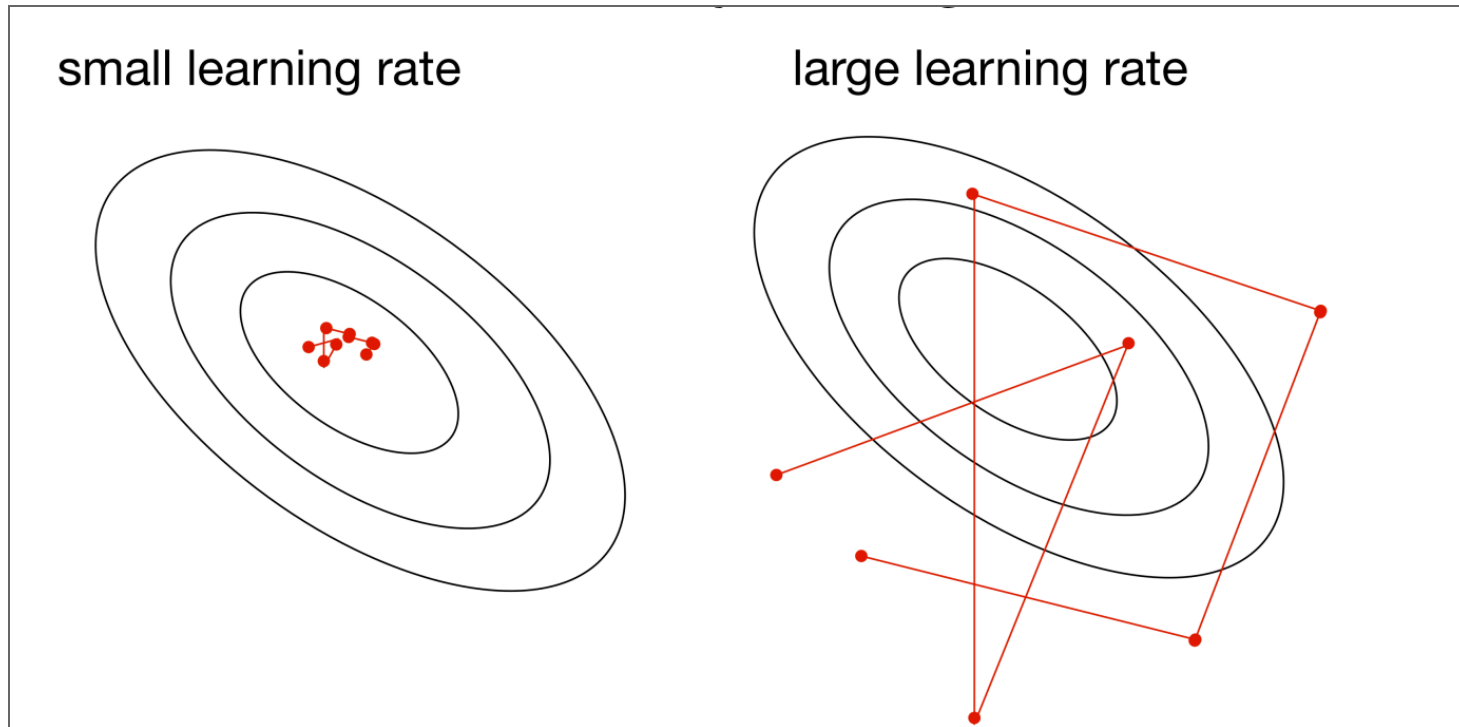
batch gradient descent



stochastic gradient descent

Learning Rates in SGD

With stochastic optimization, the learning rate also controls the size of the **fluctuations** caused by noisy gradient estimates.



Typical strategy

- Use a large learning rate early in training so you can get close to the optimum
- Gradually decay the learning rate to reduce the fluctuations

Summary

Gradient descent gives us a general way to solve optimization problems of the form

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \mathcal{J}(\mathbf{w}).$$

The basic recipe is:

initialize	→	compute gradient	→	step downhill	→	repeat
------------	---	------------------	---	---------------	---	--------

Key ideas:

- the **gradient** tells us the direction of steepest increase;
- the **negative gradient** points downhill;
- the **learning rate** determines the step size;
- convexity helps guarantee that we are moving toward a global optimum;
- batch, stochastic, and mini-batch gradient descent trade off computation and noise.

Moving Beyond Linearity

Linear Models

So far, we've mainly focused on linear models.

They are simply to describe and implement, and have advantages over other approaches in terms of interpretation and inference.

However, because the linearity assumption is almost always an approximation, linear regression can have significant limitations in terms of predictive power.

We can improve upon least squares using ridge regression and the lasso by reducing the complexity of the model (i.e. the variance of the estimates), but we are still using a linear model which can only be improved so much!

Linearity in Features vs. Parameters

Suppose $X = (X_1, X_2)$. A standard linear model is

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \varepsilon.$$

What about

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 + \varepsilon?$$

This model is **nonlinear in the features**, but still **linear in the parameters** $\boldsymbol{\beta} = (\beta_0, \dots, \beta_5)^\top$.

Why Does That Distinction Matter?

Because the model remains linear in β , we can still use familiar tools:

Estimation

- Ordinary least squares
- Maximum likelihood for a GLM

Model building

- Subset selection
- Ridge regression
- The lasso

The **linear model framework** is more flexible than a straight-line relationship.

Moving Beyond Linearity

We consider the following extensions to relax the linearity assumption in the feature space:

For one predictor ($p = 1$):

- polynomial regression;
- step functions; and
- regression splines.

For several predictors ($p > 1$):

- nearest-neighbor and local regression; and
- generalized additive models.

Polynomial Regression

Polynomial regression extends the linear model

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

to settings where the relationship between the predictors and the response is non linear by adding extra predictors, obtained by raising each of the original predictors to a power:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \epsilon_i.$$

- It is nonlinear in x , but linear in the coefficients.
- It can be fit using OLS, ridge, or the lasso.
- The coefficients themselves are **not interpretable**; we are more interested in the trend of the fitted function

$$\hat{f}(x) = \hat{\beta}_0 + \hat{\beta}_1 x + \cdots + \hat{\beta}_d x^d.$$

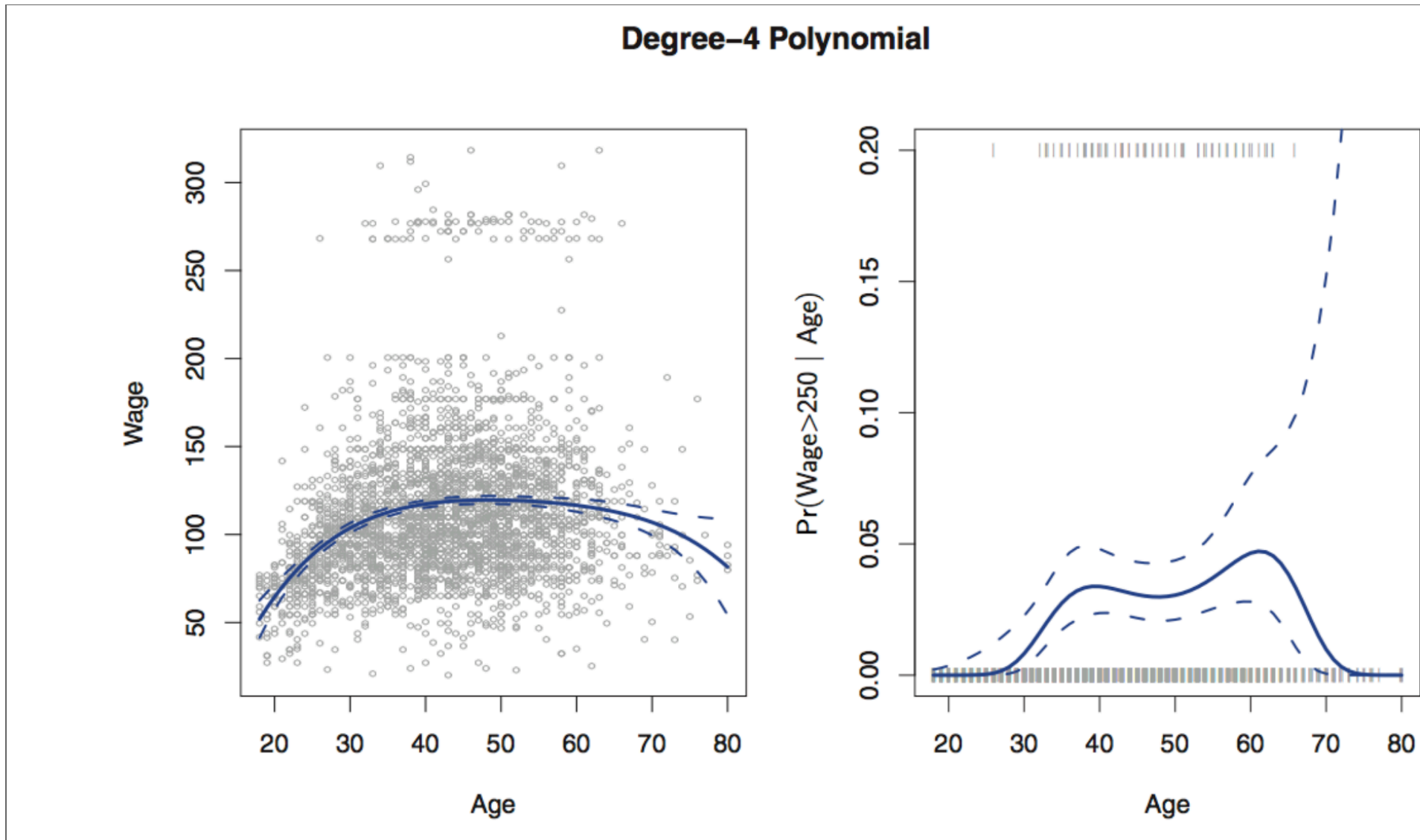
Polynomial Regression

- In practice, the degree d is usually small, often no greater than 3 or 4.
- We can choose d using cross-validation.
- Polynomial terms can also be used for classification
- For instance, in the logistic regression below (we will come back to this later), this can be fit by maximizing the likelihood

$$\text{logit}\{P(Y_i = 1 \mid X_i = x_i)\} = \beta_0 + \beta_1 x_i + \cdots + \beta_d x_i^d.$$

Note: High-degree polynomials can behave erratically near the boundaries and are particularly unreliable for extrapolation.

Polynomial Regression: Wage Data



- **Left:** The solid blue curve is a degree-4 polynomial of wage as a function of age, fit by the OLS. The dotted curves are estimated 95/% confidence intervals

- **Right:** Model the binary event $1\{\text{wage} > 250\}$ by logistic regression, with a degree-4 polynomial.

Step Functions

Polynomial regression imposes a **global** shape. A **step function** instead divides the range of X into regions.

For pre-specified K cut points $c_1 < \dots < c_K$, define

$$\begin{aligned}C_0(X) &= I(X < c_1), \\C_1(X) &= I(c_1 \leq X < c_2), \\&\vdots \\C_K(X) &= I(c_K \leq X).\end{aligned}$$

These $K + 1$ dummy variables sum to 1.

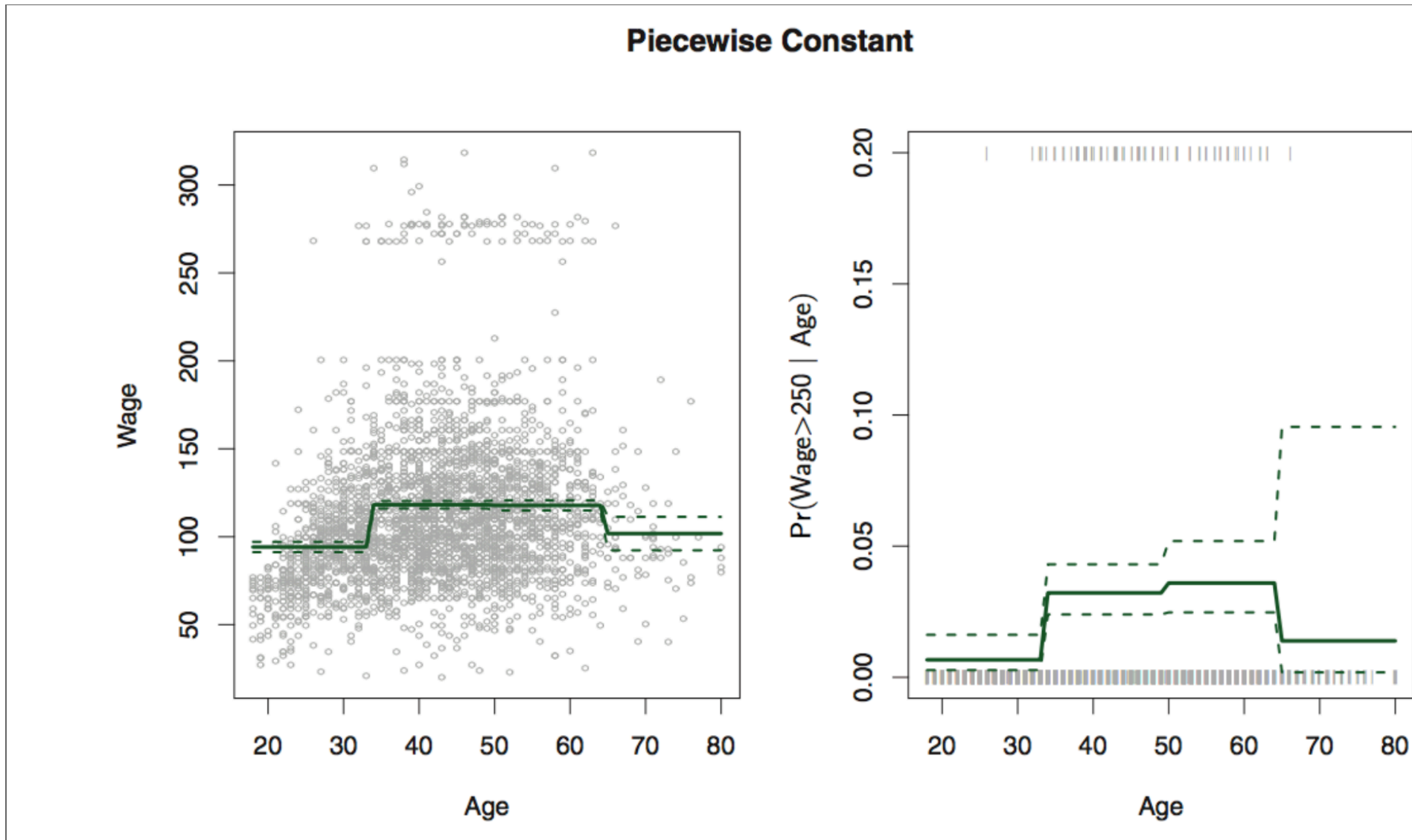
Step-Functions

Using the first interval as the reference category,

$$y_i = \beta_0 + \beta_1 C_1(x_i) + \cdots + \beta_K C_K(x_i) + \varepsilon_i.$$

- The model can be fit by OLS or a shrinkage method.
- For a given value X , at most one of $C_1, C_2, \dots, C_K$ can be non-zero.
- β_0 is the mean response in the reference interval $X < c_1$.
- β_j is the mean difference in the response Y for $c_j \leq X \leq c_{j+1}$ relative to $X < c_1$.

Step Functions: Wage Data



- **Left:** The solid blue curve is a step function of wage as a function of age, fit by least squares. The dotted curves indicate an estimated 95/% confidence interval.

- **Right:** Model the binary event $1\{\text{wage} > 250\}$ by logistic regression, with a step function.

Step Functions: Strengths and Limitations

Strengths

- Easy to fit
- Easy to explain
- Coefficients compare interpretable groups
- Common in biostatistics and epidemiology

Limitations

- The fitted relationship is discontinuous
- Important trends can be hidden
- Results can depend strongly on the cut points

Basis Functions

The models we have just discussed are all special cases of a **basis function** approach.

The idea is to have a family of functions/transformations that can be applied to a variable, X , and instead of fitting a linear model in X , we fit

$$y_i = \beta_0 + \sum_{k=1}^M \beta_k b_k(x_i) + \varepsilon_i,$$

where the $b_k(\cdot)$ are the *known* functions/transformations (we specify them ahead of time).

Model	Example basis functions
Polynomial	$b_k(x) = x^k$
Step function	$b_k(x) = C_k(x)$
Linear spline	$b_1(x) = x, b_{k+1}(x) = (x - \xi_k)_+$

Once the basis is constructed, fitting the model is just linear regression.

A very common choice for a basis function is a **regression spline**, which we discuss next.

Piecewise Polynomials

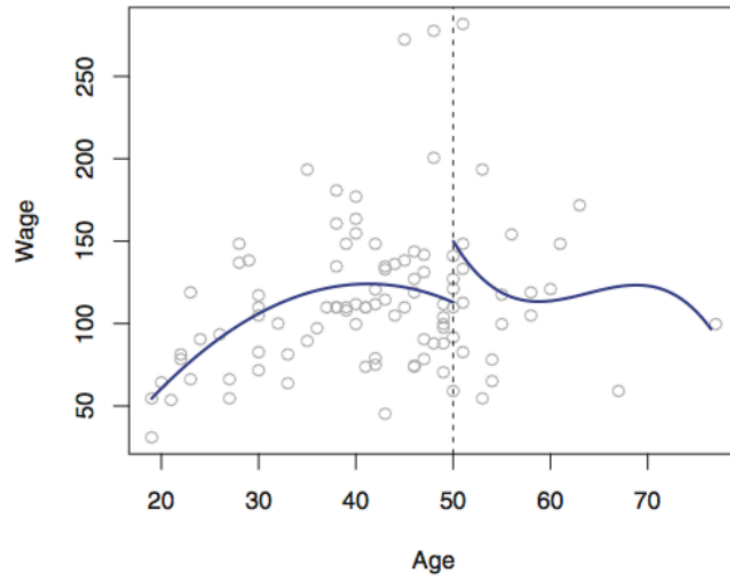
Instead of fitting one polynomial over the full range of X , fit different polynomials in different regions.

$$y_i = \begin{cases} \beta_{01} + \beta_{11}x_i + \beta_{21}x_i^2 + \beta_{31}x_i^3 + \epsilon_i, & \text{if } x_i < c, \\ \beta_{02} + \beta_{12}x_i + \beta_{22}x_i^2 + \beta_{32}x_i^3 + \epsilon_i, & \text{if } x_i \geq c. \end{cases}$$

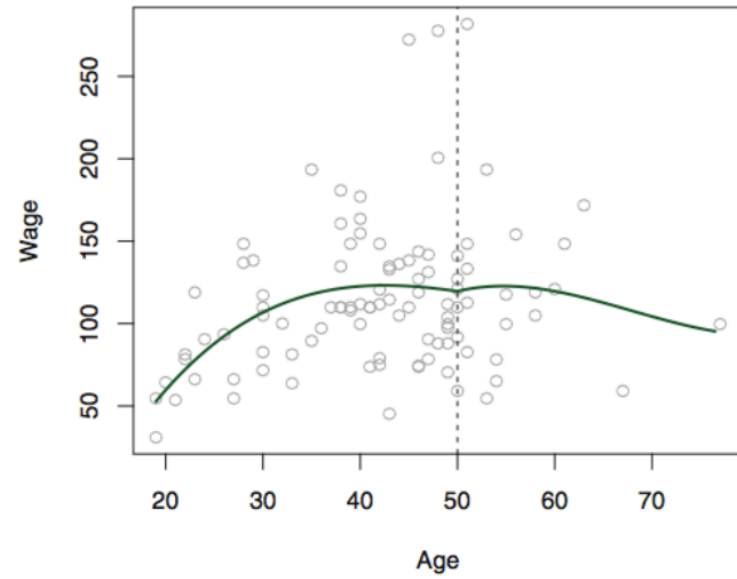
- The cut point c , i.e. where the coefficients change, is called a **knot**.
- With K knots, there are $K + 1$ regions.
- More knots allow greater local flexibility.

Piecewise Polynomials: Wage Data

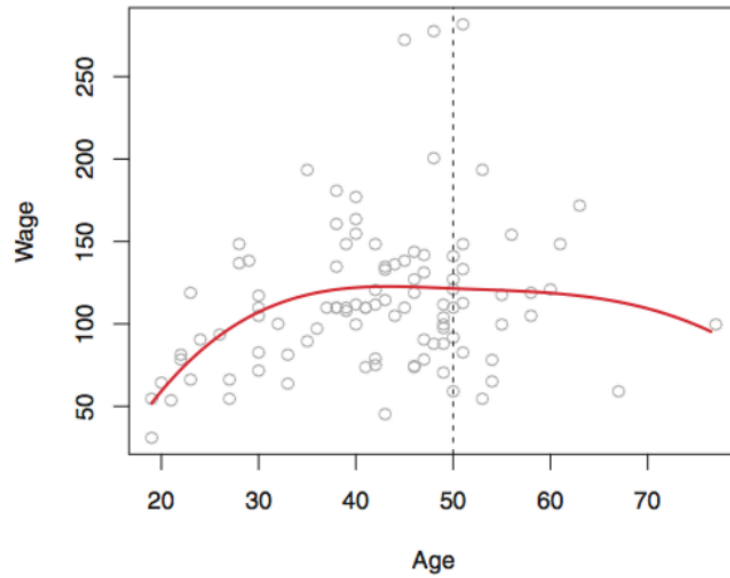
Piecewise Cubic



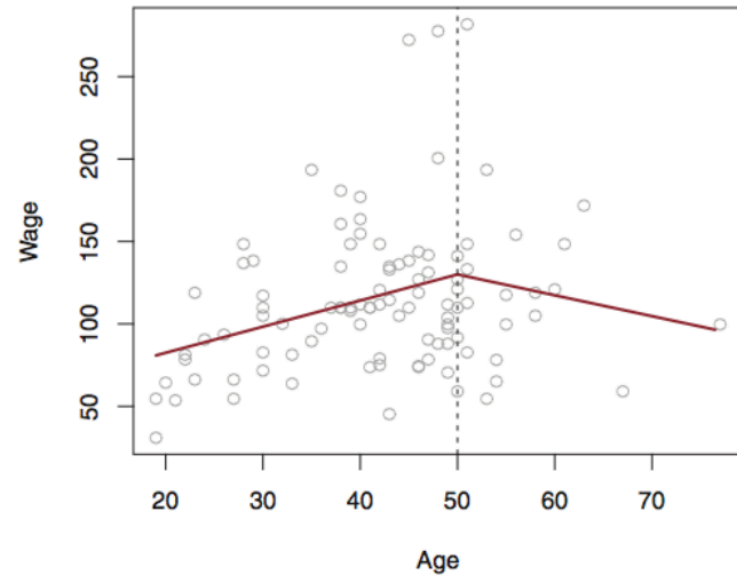
Continuous Piecewise Cubic



Cubic Spline



Linear Spline



Without *constraints* at the knots, adjacent polynomial pieces may fail to meet, or may meet with sharp corners.

Constraints and Splines

At each knot, we can impose constraints:

1. **Continuity:** the adjacent pieces have the same function value.
2. **Smoothness:** their first derivatives agree.
3. **More smoothness:** higher-order derivatives also agree.

The constrained polynomials are called **splines**.

A degree- d spline is a piecewise degree- d polynomial, with continuity in derivatives up to degree $d - 1$ at each knot.

How can we construct the degree- d spline?

Linear Splines

A **linear spline** has piecewise linear functions continuous at each knot. That is, with knots at $\xi_1 < \dots < \xi_K$,

$$y_i = \beta_0 + \beta_1 x_i + \sum_{k=1}^K \beta_{k+1} (x_i - \xi_k)_+ + \varepsilon_i,$$

where

$$(x_i - \xi_k)_+ = \begin{cases} x_i - \xi_k, & x_i > \xi_k, \\ 0, & x_i \leq \xi_k. \end{cases}$$

is called the **truncated power basis function**, and ξ_k is the knot.

Interpretation of β_1 : the averaged increase of Y associated with one unit of X for $X < \xi_1$.

Linear Splines

For example, with three knots, we start off with a basis for a linear polynomial, then add one truncated power basis function per knot:

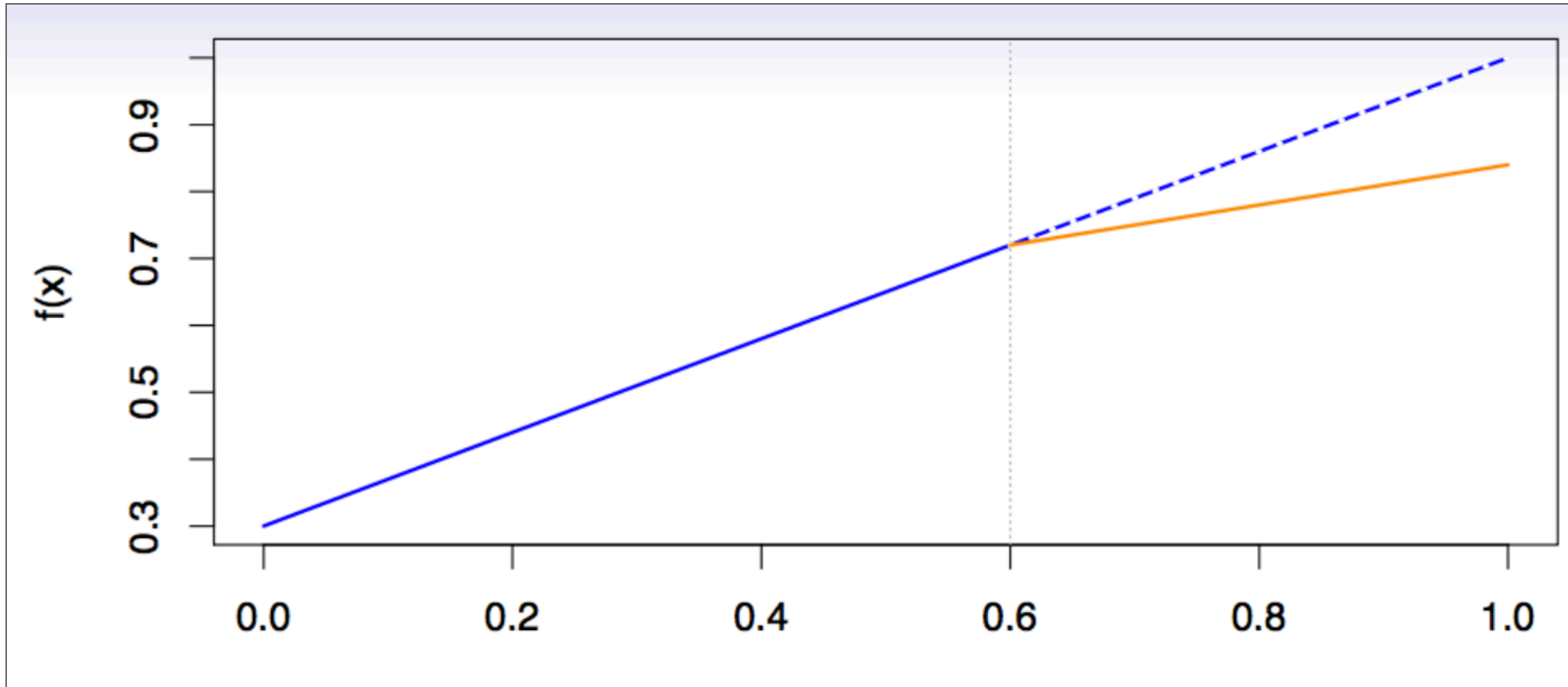
$$y = \beta_0 + \beta_1 x + \beta_2(x - \xi_1)_+ + \beta_3(x - \xi_2)_+ + \beta_4(x - \xi_3)_+.$$

Therefore, the slope in each region is

$$\begin{aligned}x &\leq \xi_1 : && \beta_1, \\ \xi_1 &< x \leq \xi_2 : && \beta_1 + \beta_2, \\ \xi_2 &< x \leq \xi_3 : && \beta_1 + \beta_2 + \beta_3, \\ x &> \xi_3 : && \beta_1 + \beta_2 + \beta_3 + \beta_4.\end{aligned}$$

Because each new term $(x - \xi_k)_+$ is zero at the knot ξ_k , the pieces meet at each knot. Thus, the fitted function is **continuous**, making it a **spline**.

How a Linear Spline Is Built



The truncated linear term is zero before the knot and grows linearly afterward. Adding it to the original line changes the slope without creating a jump.

Cubic Splines

A **cubic spline** uses piecewise cubic polynomials with continuous first and second derivatives at each knot. Again, we start off with a basis for a cubic polynomial, then add one truncated power basis function per knot.

That is, with K knots at $\xi_1 < \xi_2 < \dots < \xi_K$,

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \dots + \beta_{K+3} b_{K+3}(x_i) + \epsilon_i$$

where $b_k(\cdot)$ are basis functions

$$b_1(x) = x, \quad b_2(x) = x^2, \quad b_3(x) = x^3,$$

$$b_{k+3}(x) = (x - \xi_k)_+^3, \quad k = 1, \dots, K.$$

Cubic Splines

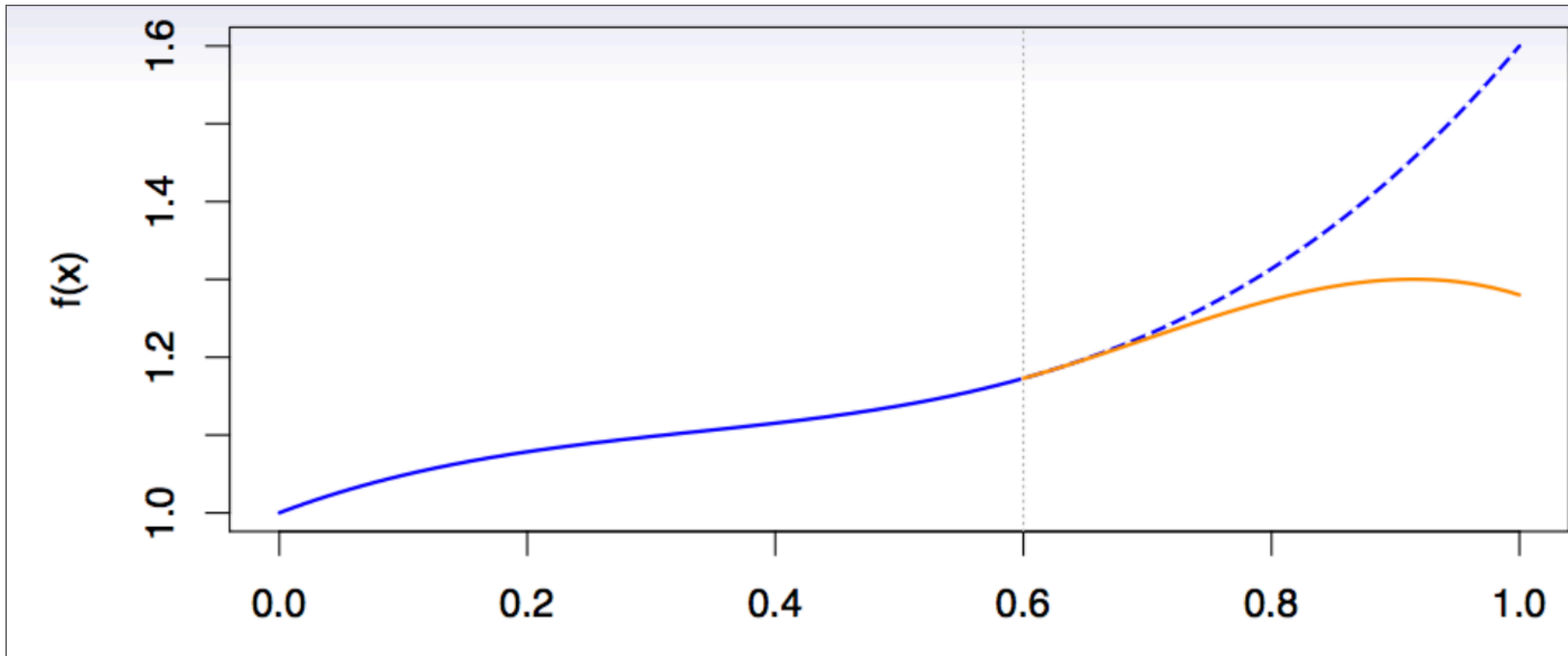
But wait.. we know a spline must have continuity in derivatives up to degree $d - 1$ at each knot.

How can we check this condition?

take derivative of the truncated power function and check continuity from both directions. Do this $d-1$ times.



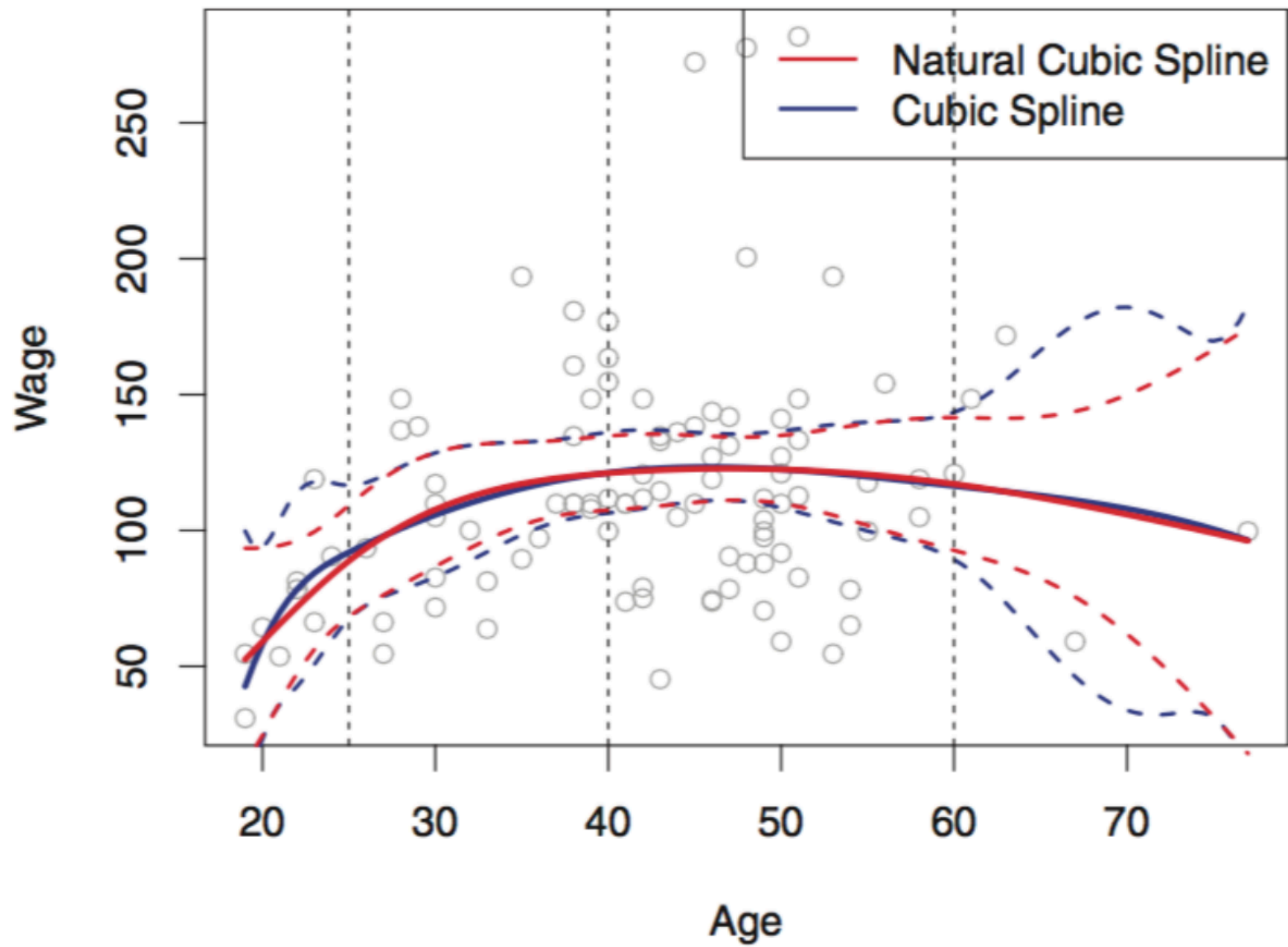
How a Cubic Spline Is Built



The truncated cubic term changes the curve only after the knot. The pieces join smoothly in value, slope, and curvature.

Natural Splines

A **natural spline** is a regression spline with additional boundary constraints: the function is required to be linear at the boundary. This reduces instability near the boundaries, where there are fewer observations and ordinary cubic splines can have high variance.



More on Splines

Choosing the number and locations of the knots

- Place knots at selected quantiles of the data or place on the range of X with equal space. Oftentimes, the placement of knots is not very crucial.
- We use cross validation to choose K

Polynomial regressions and step functions are special cases of splines

Another variant: Smoothing spline (ISLR 7.5)

What Changes When $p > 1$?

- Splines can be used for $p < 4$ (number of basis functions grows exponentially in p)
- Local approach for $p < 4$
 - nearest neighbour approach
 - local regression
- Generalized Additive Models (GAM) for large p

k -Nearest-Neighbor

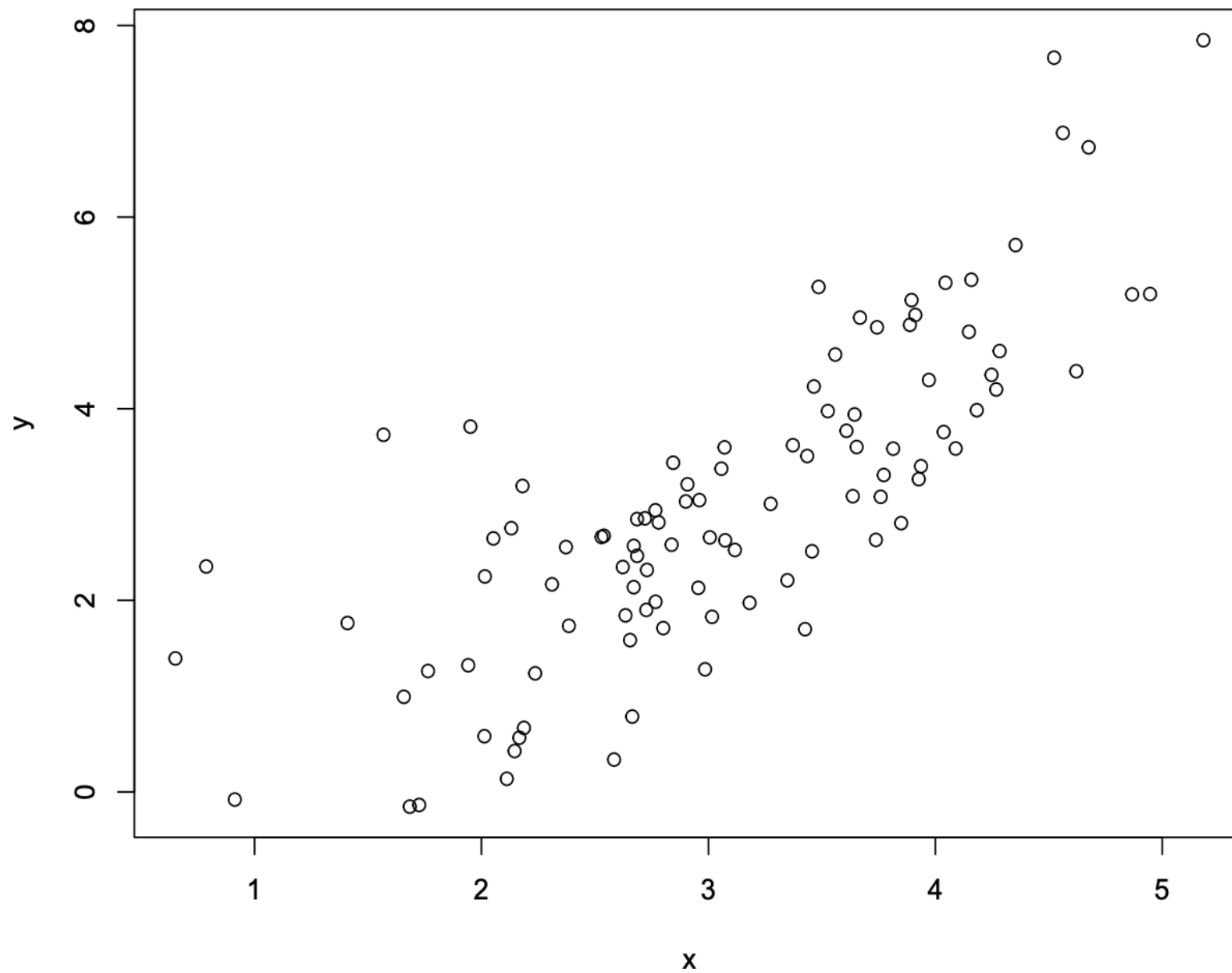
To predict at $X = x_0$:

1. Choose $k \in \{1, \dots, n\}$.
2. Find the k training points closest to x_0 ; call their index set $N_k(x_0)$.
3. Average their responses:

$$\hat{f}(x_0) = \frac{1}{k} \sum_{i \in N_k(x_0)} y_i.$$

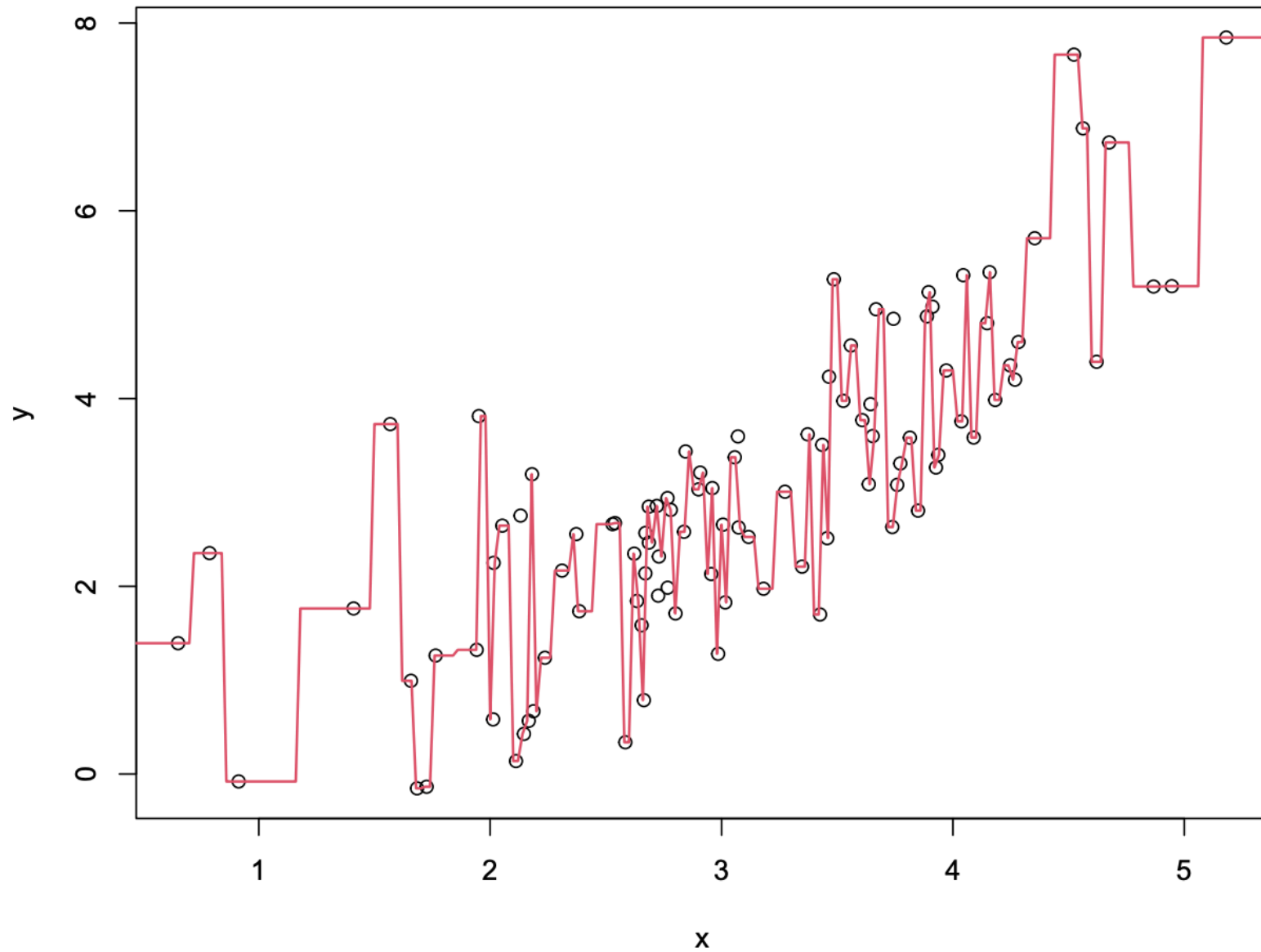
k -Nearest-Neighbor: The Role of k

100 training data points



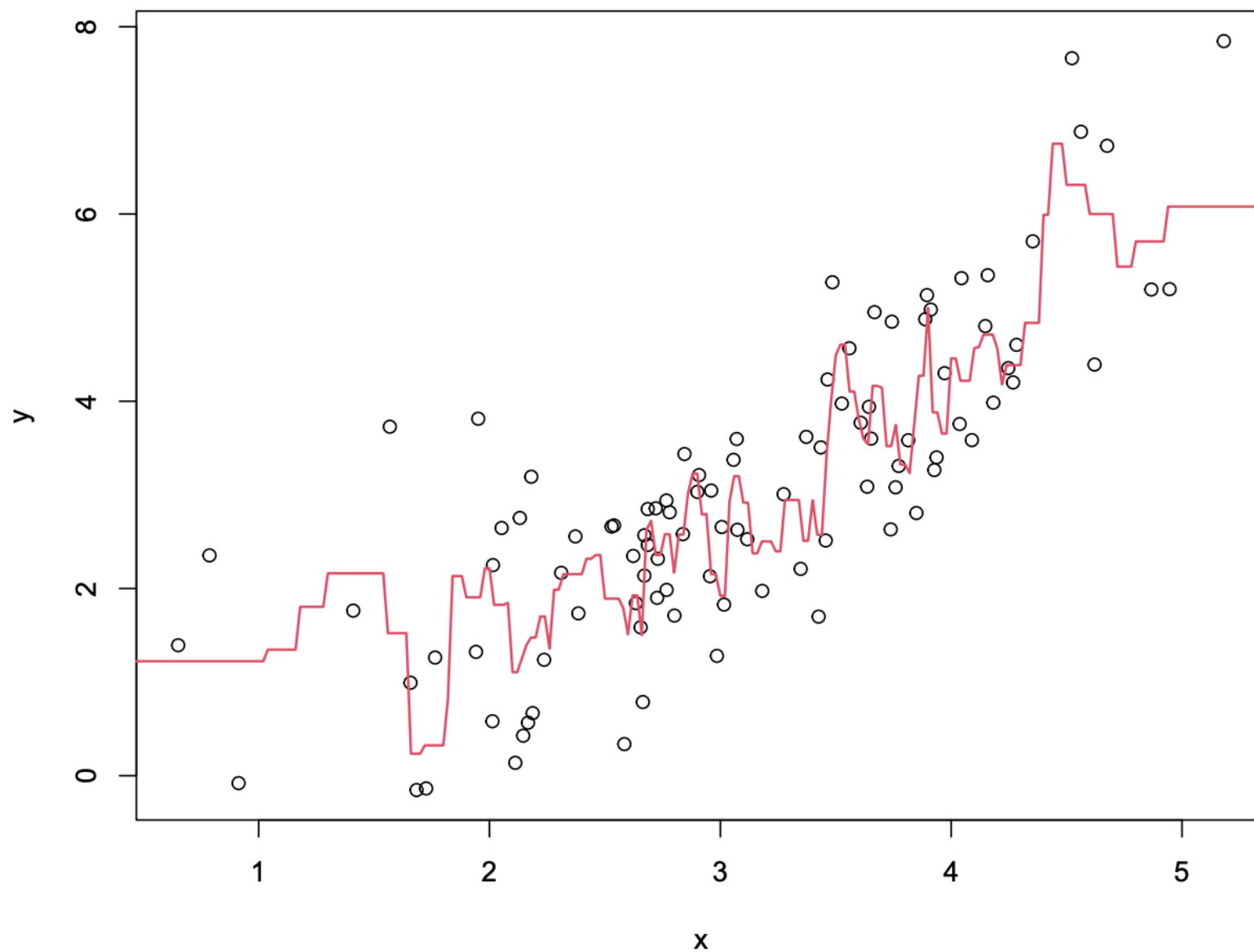
$k = 1$ Nearest Neighbour

1 nearest neighbor regression



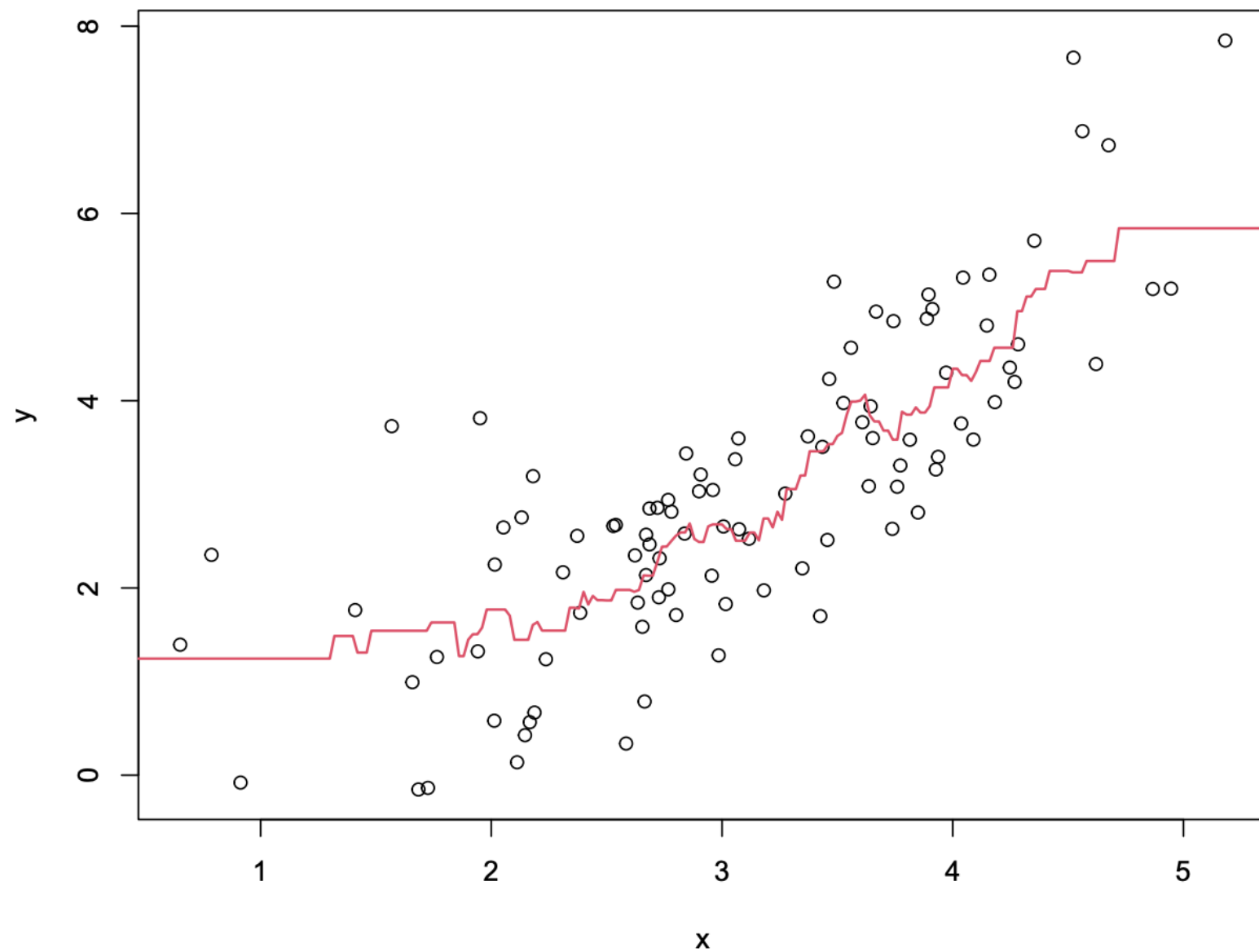
$k = 3$ Nearest Neighbour

3 nearest neighbor regression



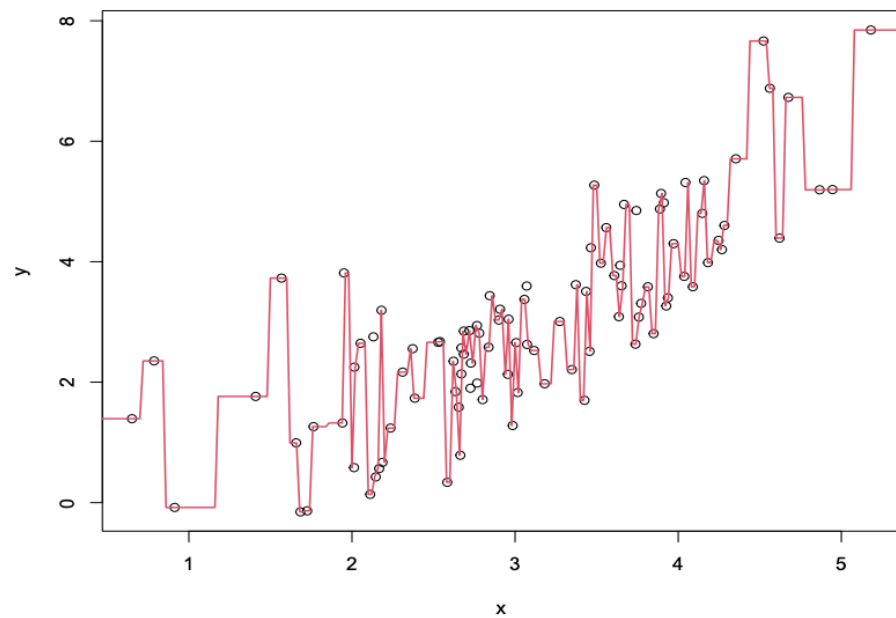
$k = 10$ Nearest Neighbour

10 nearest neighbor regression

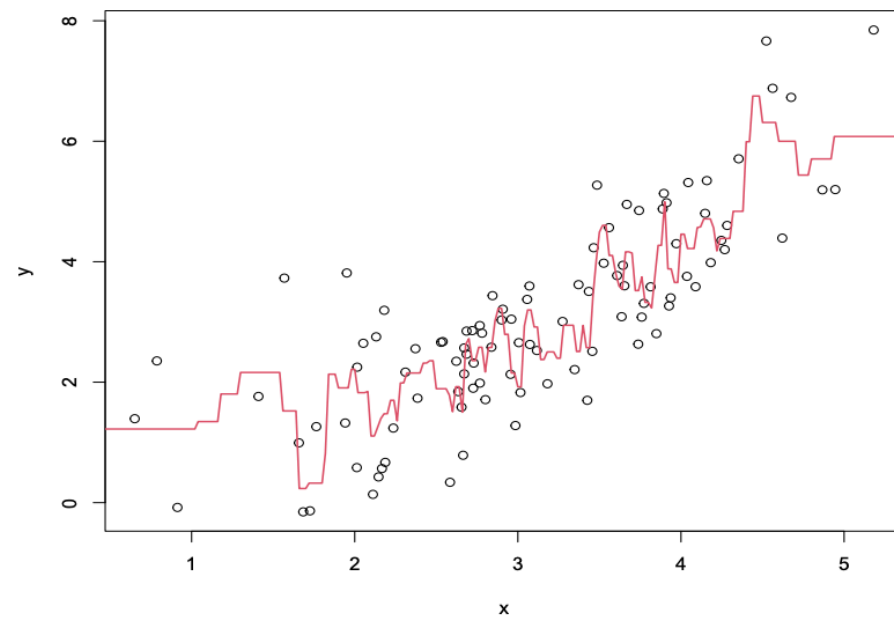


Role of k

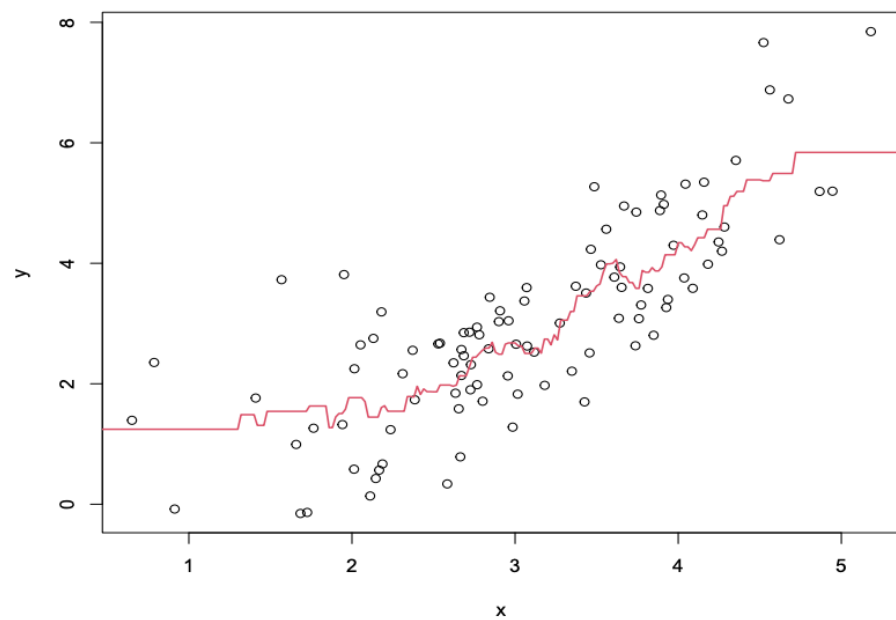
1 nearest neighbor regression



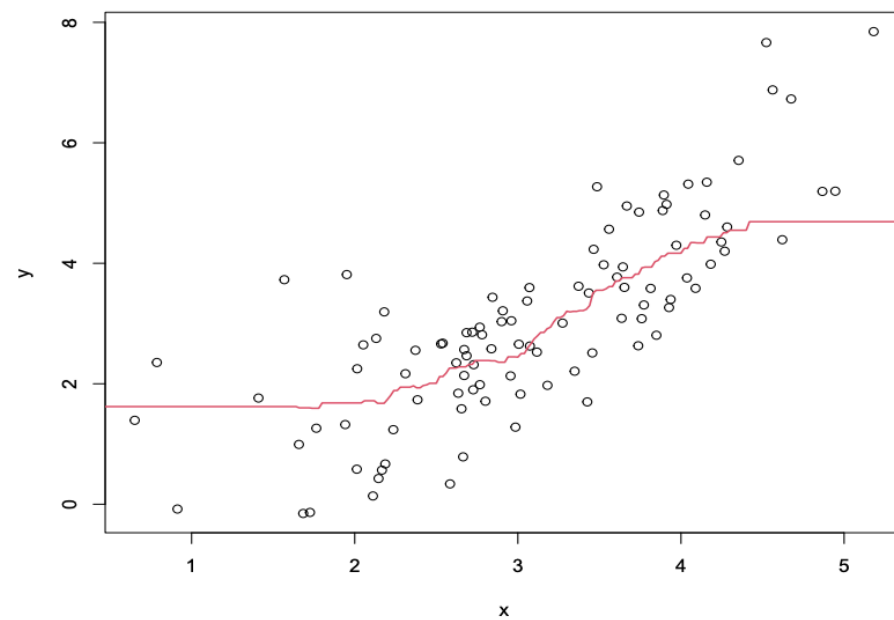
3 nearest neighbor regression



10 nearest neighbor regression



30 nearest neighbor regression



Role of k

Smaller k	Larger k
More flexible	Smoother
Lower bias	Higher bias
Higher variance	Lower variance

Choose k using **cross-validation**.

The best k is the one that gives the lowest estimated test error (not necessarily the smoothest-looking curve).

Weighted k -Nearest Neighbors

Ordinary k -NN assigns equal weight $1/k$ to every neighbor.

More generally, we can choose a weight such that

$$\hat{f}(x_0) = \sum_{i \in N_k(x_0)} K(x_i, x_0) y_i,$$

with

$$K(x_i, x_0) \geq 0, \quad \sum_{i \in N_k(x_0)} K(x_i, x_0) = 1.$$

Nearby observations can be given more influence than observations near the edge of the neighborhood.

Choice of the Weight

One popular choice is the so-called **inverse distance weighting** (IDW).
Of course there are other more sophisticated weighting schemes.

For $x_i \in N_k(x_0)$, define

$$d_i = \|x_i - x_0\|_2, \quad \text{ID}_i = \frac{1}{d_i}.$$

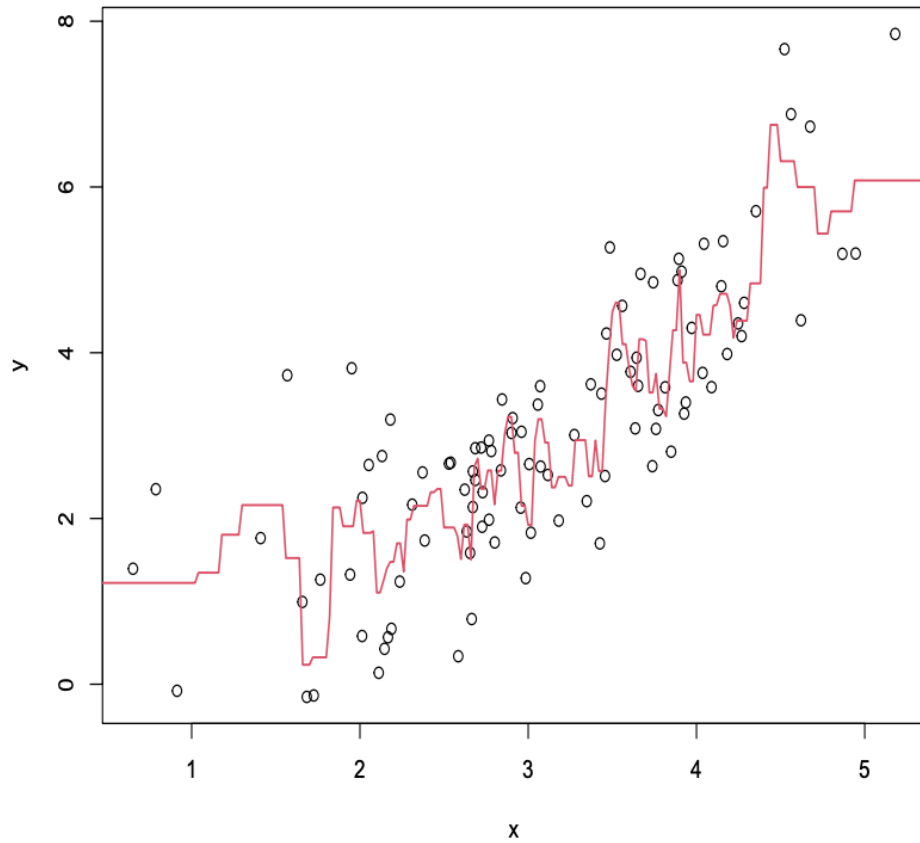
Normalize the inverse distances:

$$K(x_i, x_0) = \frac{\text{ID}_i}{\sum_{j \in N_k(x_0)} \text{ID}_j}.$$

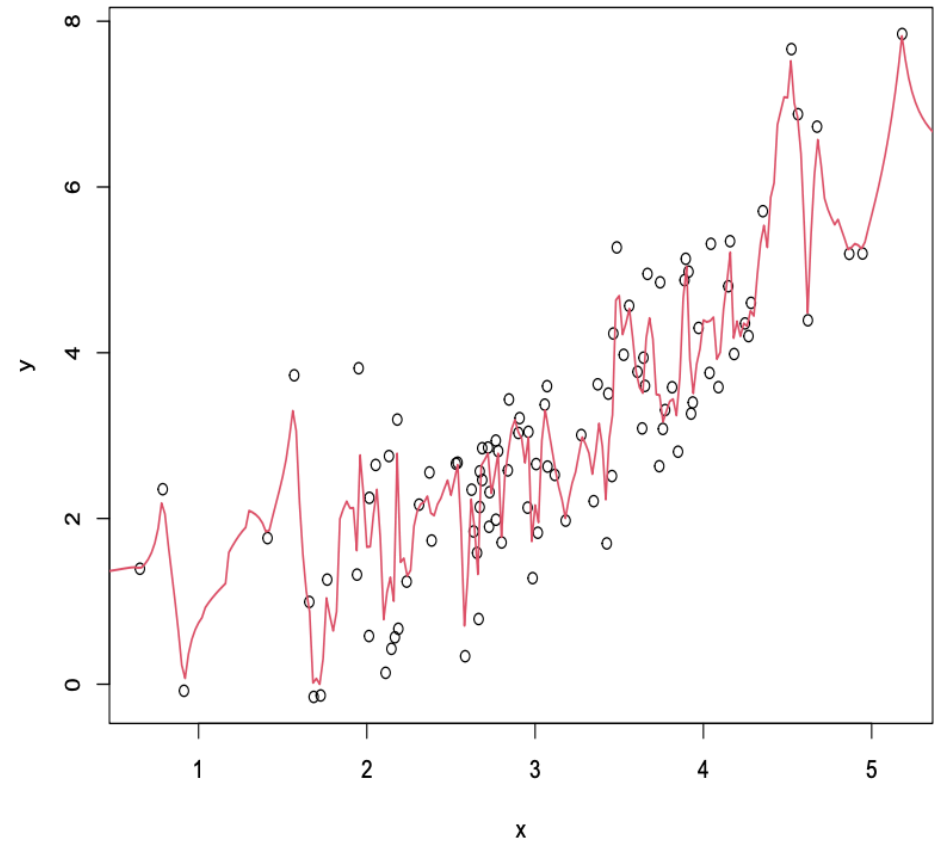
In implementation, handle $d_i = 0$ separately: if the target matches a training point exactly, that response can receive all the weight.

Weighted k NN vs k NN ($k = 3$)

3 nearest neighbor regression



3 nearest neighbor regression



Generalization of k NN: Local Regression

Recall that k NN predicts by using the local average of the **responses**

$$\hat{f}(x_0) = \frac{1}{k} \sum_{i \in N_k(x_0)} y_i.$$

Local linear regression instead fits a line near x_0 :

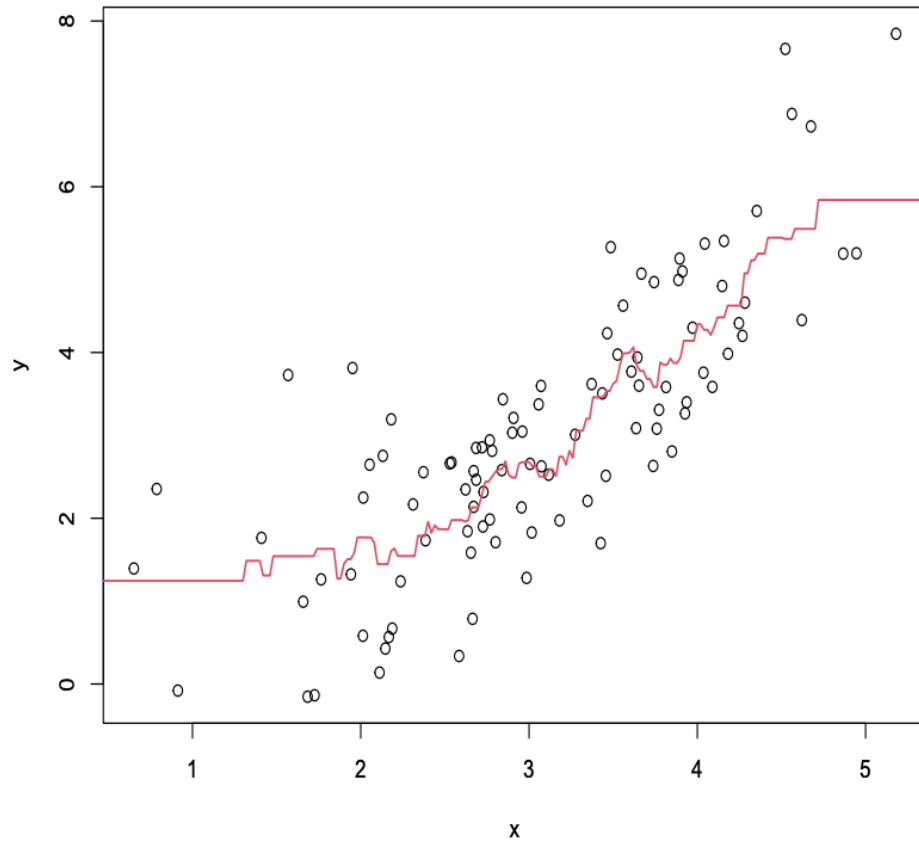
$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0,$$

where

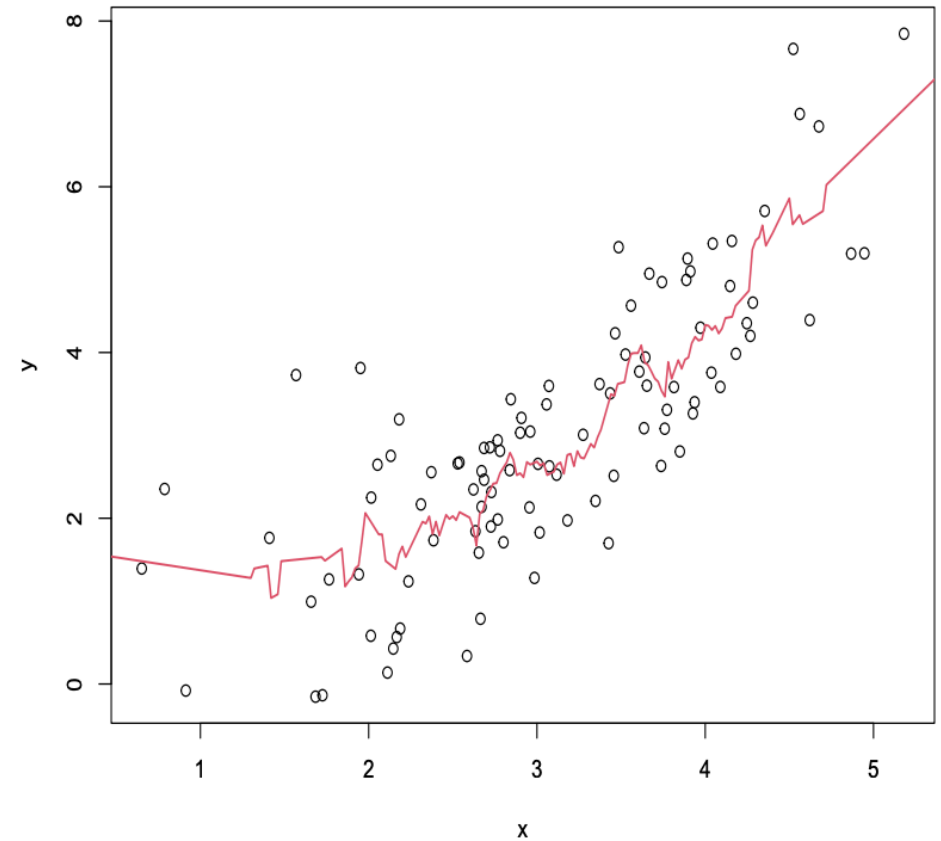
$$(\hat{\beta}_0, \hat{\beta}_1) = \arg \min_{\beta_0, \beta_1} \sum_{i \in N_k(x_0)} (y_i - \beta_0 - \beta_1 x_i)^2.$$

k -NN vs. Local Linear Regression ($k = 10$)

10 nearest neighbor regression



10 local linear regression



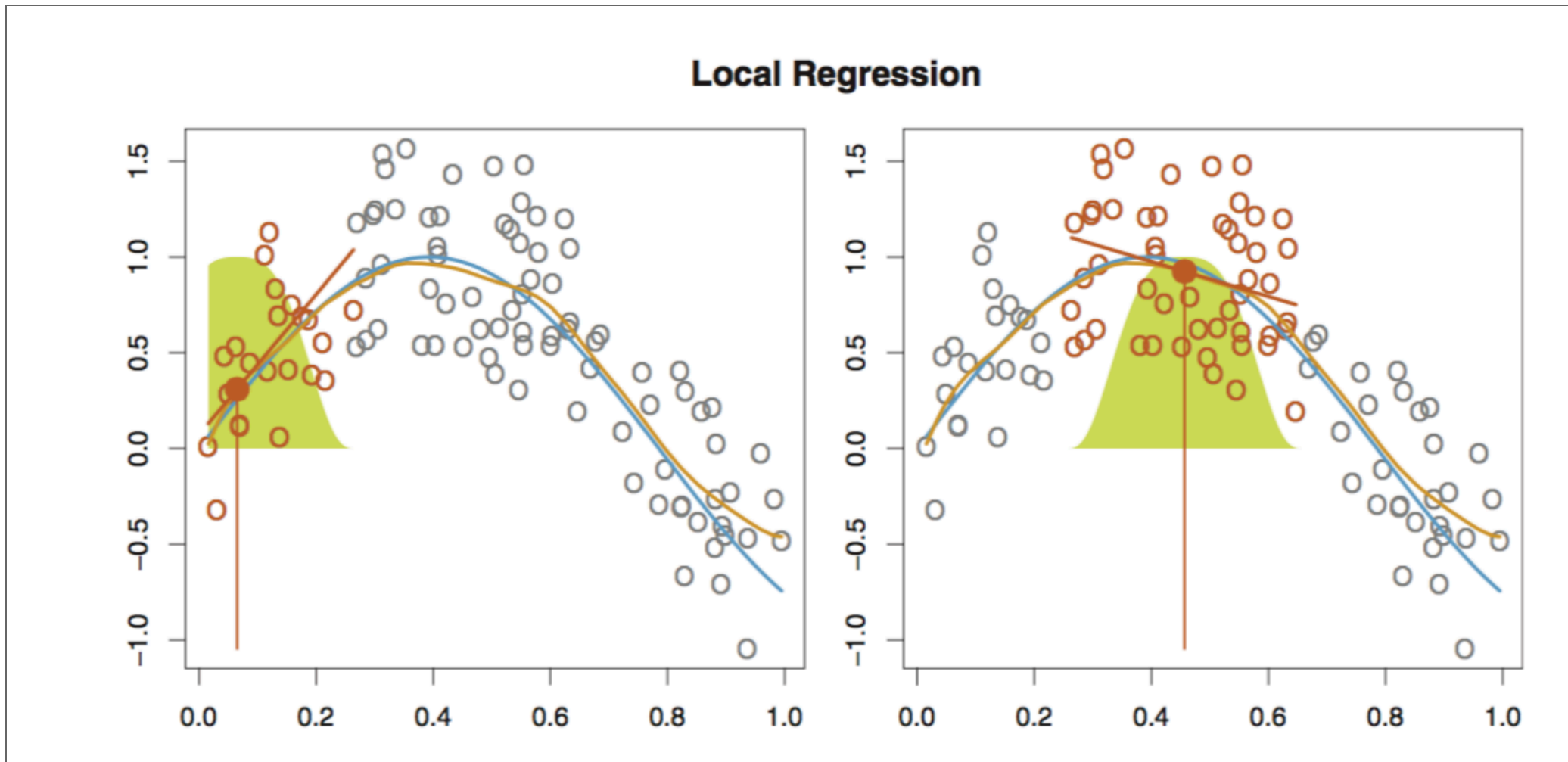
Local Regression Algorithm

1. Gather the fraction of $s = k/n$ of training points whose x_i are closest to x_0
2. Assign a weight $K_{i0} = K(x_i, x_0)$ to each point in this neighbourhood, so that the point furthest away from x_0 has weight zero, and the closest has the highest weight. All but these k nearest neighbours get weight zero.
3. Fit a *weighted least-squares regression* of the y_i on the x_i using the aforementioned weights, by finding $\hat{\beta}_0$ and $\hat{\beta}_1$ that minimize

$$\sum_{i=1}^n K_{i0} (y_i - \beta_0 - \beta_1 x_i)^2$$

4. The fitted value at x_0 is given by $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$

Simulated Example



The blue curve is true $f(x)$, and the light orange curve is the local regression $\hat{f}(x)$. The orange points are local to the target point x_0 , represented by the orange vertical line. The yellow bell-shape indicates weights assigned to each point. The fit $\hat{f}(x_0)$ at

x_0 is obtained by fitting a weighted linear regression (orange line segment), and using the fitted value at x_0 (orange solid dot) as the estimate $\hat{f}(x_0)$.

Local Regression

- The size of the neighbourhood (fraction s of training data) is a tuning parameter, which can be chosen by cross validation
- The weight of each point in the neighbourhood needs to be specified
- When we have two dimensional predictors X_1 and X_2 , we can simply use 2-dimensional neighborhoods, and fit bivariate linear regression models using the observations that are near each target point in 2-dimensional space.
- However, local regression can perform poorly if $p \geq 4$ (the curse of dimensionality)

Generalized Additive Models

A **generalized additive model** (GAM) allows a separate nonlinear function for each predictor while preserving additivity:

$$y_i = \beta_0 + f_1(x_{i1}) + \cdots + f_p(x_{ip}) + \varepsilon_i.$$

Each f_j could be:

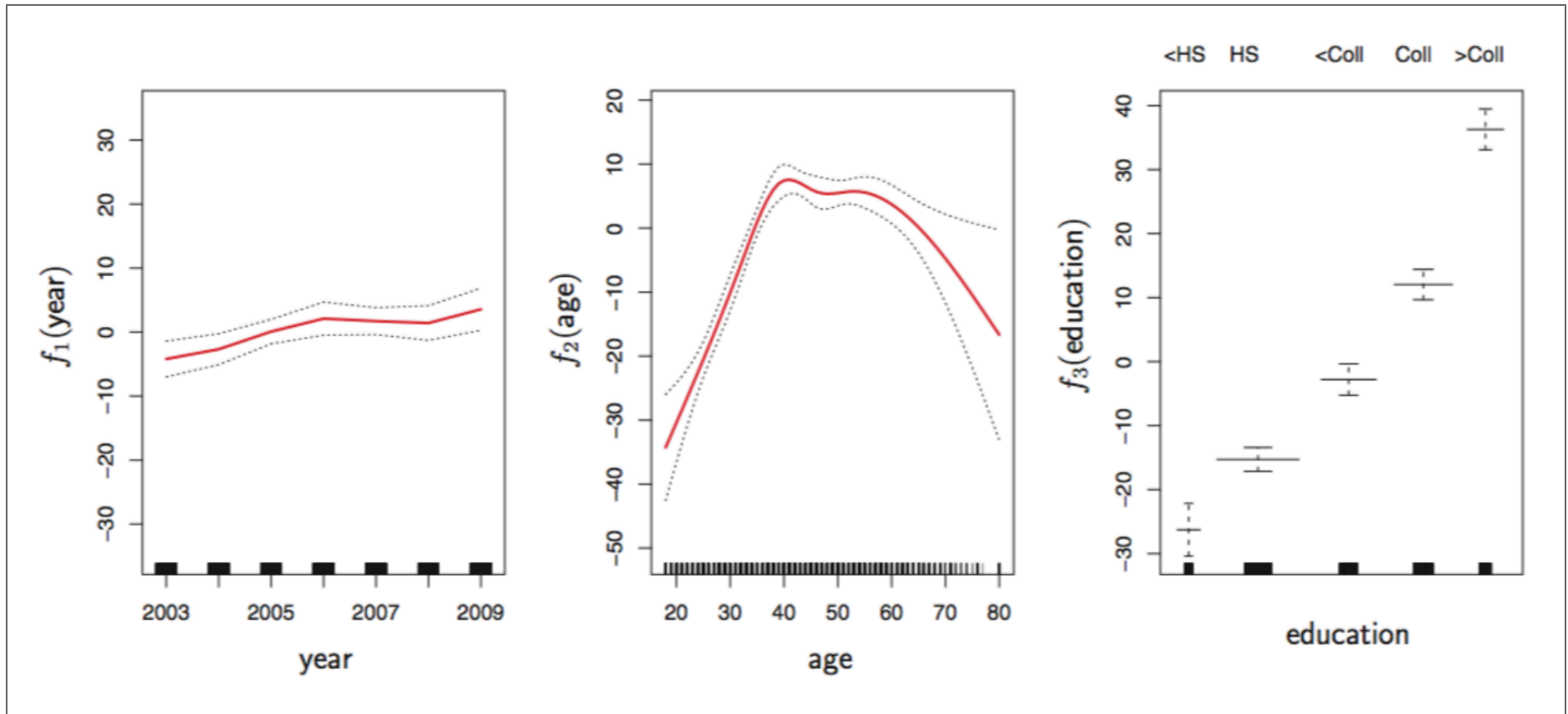
- linear;
- polynomial;
- a step function;
- a spline; or
- a local regression.

Can be applied to classification problems.

GAM: Wage Data

Consider the wage data

$$\text{wage} = \beta_0 + f_1(\text{year}) + f_2(\text{age}) + f_3(\text{education}) + \varepsilon.$$



f_1 and f_2 are natural splines; f_3 is a step function for the qualitative predictor education.

GAMs: Strengths and Limitations

- GAMs allow us to fit a non-linear function f_j to each X_j : model complicated relationship between the response and the original feature space
- The non-linear fit can potentially improve prediction accuracy
- Because the model is additive, we can still examine the effect of each X_j on Y individually while holding all of the other variables fixed.
- It avoids the curse of dimensionality by assuming additivity
- However, GAMs fail to incorporate the interaction of variables.

Summary

- **Gradient descent** provides a general iterative approach for minimizing an objective function
- The **learning rate** controls the size of each update, while convexity helps ensure that optimization leads to a global minimum.
- Linear models can represent **nonlinear relationships in the predictors** by transforming the original features.
- **Polynomial regression, step functions, and splines** are different ways of constructing flexible nonlinear relationships.
- **kNN and local regression** take a local approach:
- **GAMs** combine separate nonlinear functions of multiple predictors while retaining an additive and interpretable structure.
- The amount of flexibility in these methods is typically controlled by a **tuning parameter**, which can be selected using cross-validation.

Next Week: Classification!

Announcements: Assignment 1 due Oct. 7

Speaker notes